Elijas Dapšauskas TSf-17

In [1]:
```
import numpy as np
import pandas as pd
```

In [2]:
```
# Duomenys is skaidriu pavyzdzio (pasitikrinimui)
pvz_duomenys = """0 0 0 0
0 1 1 0
1 1 1 1
1 1 0 0
1 1 1 1"""
pvz_stulp_pavad = list('SILY')

# Duomenys is uzduoties
duomenys = """1 1 1 1 0
0 1 1 0 1
0 0 1 1 1
1 0 1 0 1
0 1 0 0 0
1 1 1 1 0
0 0 1 1 0
0 1 0 0 1
1 0 1 0 0"""
stulp_pavad = ['Namuose', 'Lietus', 'PalaikymoKom', 'Lyderiai', 'Pergalė']
```

Įkeliame duomenis

In [3]:
```
def nuskaityti_duomenis(duomenys, stulp_pavad):
    df = pd.DataFrame([k.split() for k in duomenys.split('\n')],
                      columns=stulp_pavad,
                      dtype=float)
    Y = df.iloc[:,len(df.columns)-1] # pask. stulpelis
    X = df.iloc[:,:len(df.columns)-1] # kiti stulpeliai
    return df, X, Y

# Pavyzdys
XY, X, Y = nuskaityti_duomenis(duomenys, stulp_pavad)
XY
```

Out[3]:

	Namuose	Lietus	PalaikymoKom	Lyderiai	Pergalė
0	1.0	1.0	1.0	1.0	0.0
1	0.0	1.0	1.0	0.0	1.0
2	0.0	0.0	1.0	1.0	1.0
3	1.0	0.0	1.0	0.0	1.0
4	0.0	1.0	0.0	0.0	0.0
5	1.0	1.0	1.0	1.0	0.0
6	0.0	0.0	1.0	1.0	0.0
7	0.0	1.0	0.0	0.0	1.0

8	1.0	0.0	1.0	0.0	0.0

Apskaičiuojame gini koeficientus ir surikiuojame juos didėjančia tvarka. Pirmoji šaka skiriama pagal ties mažiausią koeficientą turinčiu stulpeliu ir t.t.

In [4]:

```
def apskaiciuoti_gini(x):
    Pr_s0 = sum(x==0) / len(x)
    Pr_s1 = sum(x==1) / len(x)
    Pr_y0_s0 = sum(Y.loc[x==0]==0) / sum(x==0)
    Pr_y1_s0 = sum(Y.loc[x==0]==1) / sum(x==0)
    Pr_y0_s1 = sum(Y.loc[x==1]==0) / sum(x==1)
    Pr_y1_s1 = sum(Y.loc[x==1]==1) / sum(x==1)
    gini_impurity_s0 = 1 - Pr_y0_s0**2 - Pr_y1_s0**2
    gini_impurity_s1 = 1 - Pr_y0_s1**2 - Pr_y1_s1**2
    gini_impurity_s0, gini_impurity_s1
    gini_s = Pr_s0 * gini_impurity_s0 + Pr_s1 * gini_impurity_s1
    return(gini_s)

# Pavyzdys
X.apply(apskaiciuoti_gini).sort_values()
```

Out[4]:

```
Namuose         0.433333
Lyderiai        0.433333
Lietus          0.488889
PalaikymoKom    0.492063
dtype: float64
```

Pirmiausia sukursime funkciją kuri keliauja per grafą.

In [5]:

```
def keliauti_per_grafa(stulpeliai, ar_baigti_paieska, _istorija=None):
    _istorija = _istorija or []
    if ar_baigti_paieska(_istorija) or not len(stulpeliai):
        return
    for v in (0, 1):
        nauja_istorija = _istorija.copy()
        nauja_istorija.append((stulpeliai[0], v))
        keliauti_per_grafa(stulpeliai[1:],
                           ar_baigti_paieska,
                           nauja_istorija)

## Pavyzdys
# def ar_baigti_paieska(istorija):
#     if len(istorija) == 3:
#         print(istorija)
#         return True
#     return False
#keliauti_per_grafa(['A', 'B', 'C'], ar_baigti_paieska)
```

Tada sukursime funkciją, kuri tikrina, ar tam tikras eilučių filtravimas leidžia pasiekti homogenišką Y stulpelį (t.y. atfiltravus visi Y=1 arba visi Y=0).

In [6]:

```
def spausdinti_atsakyma(filtrai, y0, tikimybe=None):
    salyga = ' ir '.join(
        f'[{k} = {"Taip" if v else "Ne"}]' for k, v in filtrai)
    txt = (
        f'Jei {salyga}:\n  '
```

```
        f'tada Y = {int(y0) if tikimybe != 0.5 else "0 arba 1"}'
    )
    if tikimybe:
        txt = f"{txt}\n  (tikimybė: {100*tikimybe:.2f}%)"
    print(f'{txt}\n')

def filtruoti(df, filtrai):
    for stulp, reiksme in filtrai:
        df = df.loc[df[stulp] == reiksme]
    return df

def trumpiausia_filtru_israiska(XY, filtrai):
    for i in reversed(range(len(filtrai))):
        if (filtruoti(XY, filtrai[:i]).shape[0]
            != filtruoti(XY, filtrai[:i+1]).shape[0]):
            return filtrai[:i+1]

def ar_Y_homogeniskas(XY, filtrai, viso_stulpeliu):
    Y = filtruoti(XY, filtrai).iloc[:,-1]
    if not len(Y):
        return False
    y0 = Y.iloc[0]
    if not (y0 == Y).all():
        if len(filtrai) == viso_stulpeliu:
            spausdinti_atsakyma(
                trumpiausia_filtru_israiska(XY, filtrai),
                y0, np.mean(y0 == Y)
            )
        return False
    spausdinti_atsakyma(trumpiausia_filtru_israiska(XY, filtrai), y0)
    return True

## Pavyzdys
#filtrai = [('S', 1), (('I'), 1)]
#filtruoti(XY, filtrai), ar_Y_homogeniskas(XY, filtrai)
```

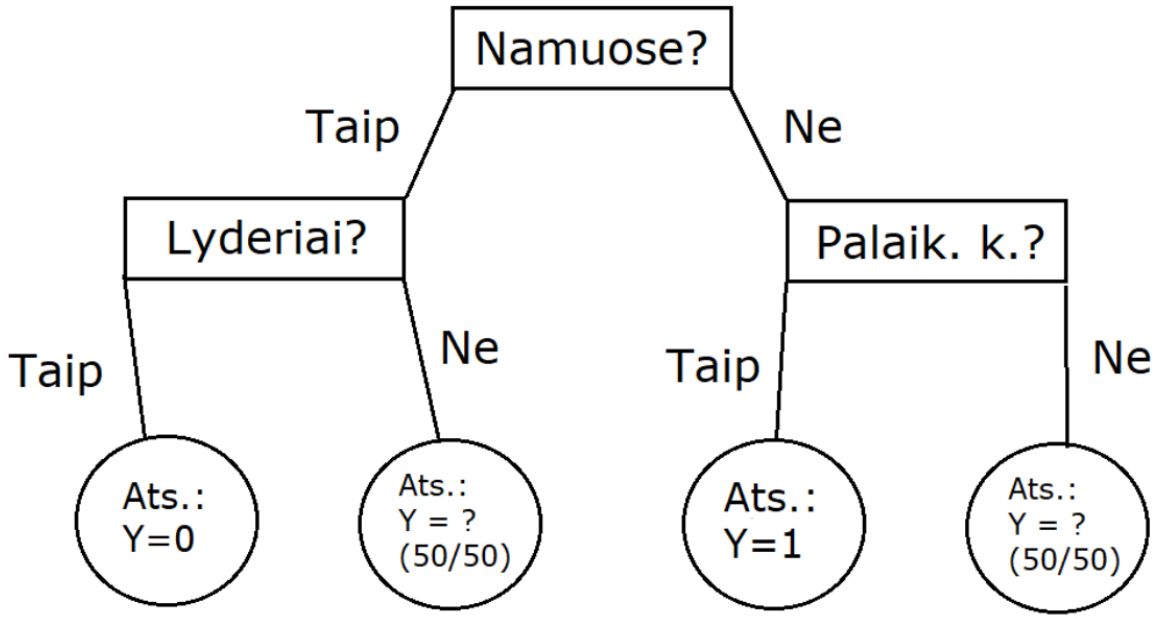
Apjungiame praeitas dvi funkcijas.

In [7]:

```
XY, X, Y = nuskaityti_duomenis(duomenys, stulp_pavad)
stulpeliu_pav = X.apply(apskaiciuoti_gini).sort_values().index
keliauti_per_grafa(
    stulpeliu_pav,
     lambda istorija: ar_Y_homogeniskas(XY, istorija, len(stulpeliu_pav))
)
```

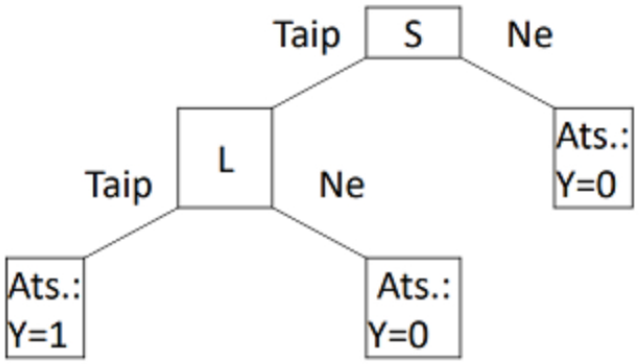
```
Jei [Namuose = Ne] ir [Lyderiai = Ne] ir [Lietus = Taip] ir [PalaikymoKom = Ne]:
  tada Y = 0 arba 1
  (tikimybė: 50.00%)

Jei [Namuose = Ne] ir [Lyderiai = Ne] ir [Lietus = Taip] ir [PalaikymoKom = Taip]:
  tada Y = 1

Jei [Namuose = Ne] ir [Lyderiai = Taip]:
  tada Y = 0 arba 1
  (tikimybė: 50.00%)

Jei [Namuose = Taip] ir [Lyderiai = Ne]:
  tada Y = 0 arba 1
  (tikimybė: 50.00%)
```

```
Jei [Namuose = Taip] ir [Lyderiai = Taip]:
  tada Y = 0
```



Įsitikiname, kad kodas veikia teisingai išspręsdami pavyzdinį uždavinį:

In [8]:

```python
XY, X, Y = nuskaityti_duomenis(pvz_duomenys, pvz_stulp_pavad)
stulpeliu_pav = X.apply(apskaiciuoti_gini).sort_values().index
keliauti_per_grafa(
    stulpeliu_pav,
    lambda istorija: ar_Y_homogeniskas(XY, istorija, len(stulpeliu_pav))
)
```

```
Jei [S = Ne]:
  tada Y = 0

Jei [S = Taip] ir [L = Ne]:
  tada Y = 0

Jei [S = Taip] ir [L = Taip]:
  tada Y = 1
```